

Explicação do Código com Trechos

1. Importação das bibliotecas

```
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import linear_kernel
```

 O que isso faz:

- pandas: usado para ler e organizar os dados (tipo uma planilha)
 - TfidfVectorizer: transforma texto em números
 - linear_kernel: calcula o quanto dois filmes são parecidos
-

2. Criação da classe

```
class RecomendadorFilmes:
```

 Aqui estamos criando um “sistema organizado”.

 Em vez de código solto, criamos uma classe que representa o recomendador.

3. Função de inicialização

```
def __init__(self, caminho_arquivo):
```

 Essa função roda automaticamente quando criamos o sistema.

Exemplo:

```
sistema = RecomendadorFilmes("filmes.csv")
```

4. Leitura do arquivo

```
self.df = pd.read_csv(
    caminho_arquivo,
    delimiter=',',
    encoding='utf-8',
    engine='python',
```

```
on_bad_lines='skip')
)
```

 O que acontece aqui:

- Lê o arquivo CSV
 - Cria uma tabela (DataFrame)
 - Ignora linhas com erro
-

5. Informações da base

```
print(f"\nLinhas: {self.df.shape[0]}")
print(f"Colunas: {self.df.shape[1]}")
```

 Mostra:

- Quantidade de filmes
 - Quantidade de colunas
-

6. Seleção de colunas

```
self.df = self.df[['original_title', 'overview']]
```

 Mantém apenas:

- Nome do filme
- Sinopse

 Remove o resto para simplificar

7. Remover dados vazios

```
self.df = self.df.dropna(subset=['overview']).reset_index(drop=True)
```

 Remove filmes sem descrição

 Porque sem sinopse não dá pra comparar

8. Padronização dos títulos

```
self.df['original_title_clean'] = self.df['original_title'].str.lower().str.strip()
```

🔍 Faz duas coisas:

- Deixa tudo minúsculo
- Remove espaços extras

👉 Evita erro quando o usuário digita

📌 9. Definir o conteúdo

```
self.df['content'] = self.df['overview']
```

🔍 Define o que será usado para comparar os filmes

👉 Aqui usamos só a sinopse

📌 10. Configurar TF-IDF

```
self.tfidf = TfidfVectorizer(  
    stop_words='english',  
    ngram_range=(1, 2),  
    max_df=0.8,  
    min_df=5  
)
```

🔍 O que cada parte faz:

- stop_words='english' → remove palavras comuns
- ngram_range=(1,2) → pega palavras e combinações
- max_df=0.8 → ignora palavras muito repetidas
- min_df=5 → ignora palavras raras

👉 Isso melhora a qualidade da análise

📌 11. Criar matriz numérica

```
self.tfidf_matrix = self.tfidf.fit_transform(self.df['content'])
```

 Aqui acontece a mágica:

 Cada filme vira uma sequência de números

12. Calcular similaridade

```
self.cosine_sim = linear_kernel(self.tfidf_matrix, self.tfidf_matrix)
```

 Compara todos os filmes entre si

 Resultado:

- matriz de similaridade
-

13. Criar índice de busca

```
self.indices = pd.Series(  
    self.df.index,  
    index=self.df['original_title_clean']  
)  
.drop_duplicates()
```

 Cria um “dicionário”:

"batman" → 123

"avatar" → 45

 Isso permite encontrar o filme rapidamente

FUNÇÃO DE RECOMENDAÇÃO

14. Função principal

```
def recomendar(self, titulo, top_n=5):
```

 Recebe:

- nome do filme
 - quantidade de recomendações
-

15. Normalizar entrada

```
titulo = titulo.lower().strip()
```

👉 Garante que o sistema entenda o que o usuário digitou

📌 16. Verificar existência

```
if titulo not in self.indices:
```

👉 Se o filme não existir, avisa o usuário

📌 17. Pegar índice do filme

```
idx = self.indices[titulo]
```

👉 Encontra a posição do filme na base

📌 18. Calcular similaridade

```
sim_scores = list(enumerate(self.cosine_sim[idx]))
```

👉 Compara o filme escolhido com todos os outros

📌 19. Ordenar resultados

```
sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
```

👉 Do mais parecido → menos parecido

📌 20. Ignorar o próprio filme

```
sim_scores = sim_scores[1:top_n + 1]
```

👉 O primeiro sempre é ele mesmo

📌 21. Pegar índices finais

```
movie_indices = [i[0] for i in sim_scores]
```

👉 Guarda os melhores filmes

📌 22. Mostrar resultado

for i, idx in enumerate(movie_indices, start=1):

👉 Loop para mostrar os filmes

```
print(f"{i}. {titulo_filme}")
```

```
print(f" {overview[:120]}...\n")
```

👉 Mostra:

- nome do filme
- parte da sinopse